



Prompt Eng-2_Common Application Patterns & Chatbot

📁 種類	Prompt Engineering
⚙️ Status	整理中
📖 課程/書籍名稱	Prompt Engineering - Deeplearning.ai

▼ Table of Content

LLM 的常見應用模式 (Common LLM Application Patterns)

模式一：摘要 (Summarizing)

通用摘要

特定焦點、對象摘要

資訊提取 (Extracting) vs. 摘要 (Summarizing) :

模式二：推斷 (Inferring)

情感分析 (Sentiment Analysis)

情緒識別 (Emotion Recognition)

資訊提取與結構化

主題推斷 (Topic Inference)

模式三：轉換 (Transforming)

語言翻譯

語氣轉換

格式轉換

拼寫與語法校對

模式四：擴寫 (Expanding)

temperature 參數

範例：自動生成回覆郵件

打造客製化 Chatbot (Practical Application: Building a Custom Chatbot)

Chat Completions API 的核心訊息結構

對話歷史 (Context) 的重要性

設定 System Message

管理對話歷史 (Context)

LLM 的常見應用模式 (Common LLM Application Patterns)

本章節將探討課程中提及的幾種核心應用能力：**摘要 (Summarizing)**、**推斷 (Inferring)**、**轉換 (Transforming)** 和 **擴寫 (Expanding)**。這些能力如同基礎的建構模組，是打造多樣化 AI 應用的基石。

模式一：摘要 (Summarizing)

摘要任務的核心是從長篇文本中提煉出關鍵資訊，幫助使用者快速掌握核心內容。

範例：熊貓絨毛玩具評論

```
prod_review = """
Got this panda plush toy for my daughter's birthday, \
who loves it and takes it everywhere. It's soft and \
super cute, and its face has a friendly look. It's \
a bit small for what I paid though. I think there \
might be other options that are bigger for the \
same price. It arrived a day earlier than expected, \
so I got to play with it myself before I gave it \
to her.
"""
```

如何透過調整 Prompt 來實現不同目的的摘要：

通用摘要

生成一個簡短、全面的摘要，適合快速瀏覽。

```
prompt = f"""
Your task is to generate a short summary of a product \
review from an ecommerce site.
```

Summarize the review below, delimited by triple backticks, in at most 30 words.

```
Review: ```{prod_review}```  
"""
```

```
response = get_completion(prompt)  
print(response)
```

Soft, cute panda plush toy loved by daughter, but smaller than expected for the price. Arrived early, friendly face.

特定焦點、對象摘要

目的： 指示模型專注於與特定方面相關的資訊。

- 為「Shipping Department」生成的摘要

```
prompt = f"""  
Your task is to generate a short summary of a product \  
review from an ecommerce site to give feedback to the \  
Shipping department.  
  
Summarize the review below, delimited by triple  
backticks, in at most 30 words, and focusing on any aspects \  
that mention shipping and delivery of the product.  
  
Review: ```{prod_review}```  
"""  
  
response = get_completion(prompt)  
print(response)
```

The customer was pleased with the early delivery of the panda plush toy, but felt it was slightly small for the price paid.

- 為「Pricing Department」生成的摘要

```
prompt = f"""
Your task is to generate a short summary of a product \
review from an ecommerce site to give feedback to the \
pricing department, responsible for determining the \
price of the product.

Summarize the review below, delimited by triple
backticks, in at most 30 words, and focusing on any aspects \
that are relevant to the price and perceived value.

Review: ``{prod_review}``
"""

response = get_completion(prompt)
print(response)
```

The panda plush toy is loved for its softness and cuteness, but some customers feel it's a bit small for the price.

資訊提取 (Extracting) vs. 摘要 (Summarizing) :

目的：與進行「摘要」不同，此 Prompt 要求模型僅「提取」與特定主題直接相關的句子，輸出更為原始和聚焦的資訊。

- 提取給 Shipping Apartment 的 Feedback

```
prompt = f"""
Your task is to extract relevant information from \
a product review from an ecommerce site to give \
feedback to the Shipping department.
```

```
From the review below, delimited by triple quotes \
extract the information relevant to shipping and \
delivery. Limit to 30 words.
```

```
Review: ``{prod_review}``
"""
```

```
response = get_completion(prompt)
print(response)
```

Feedback: The product arrived a day earlier than expected, allowing the customer to play with it before giving it as a gift.

- 從多個評論分別提取給 Pricing Apartment Feedback

```
review_1 = prod_review
```

```
# review for a standing lamp
```

```
review_2 = """
```

```
Needed a nice lamp for my bedroom, and this one \
had additional storage and not too high of a price \
point. Got it fast - arrived in 2 days. The string \
to the lamp broke during the transit and the company \
happily sent over a new one. Came within a few days \
as well. It was easy to put together. Then I had a \
missing part, so I contacted their support and they \
very quickly got me the missing piece! Seems to me \
to be a great company that cares about their customers \
```

and products.

"""

review for an electric toothbrush

review_3 = """

My dental hygienist recommended an electric toothbrush, \ which is why I got this. The battery life seems to be \ pretty impressive so far. After initial charging and \ leaving the charger plugged in for the first week to \ condition the battery, I've unplugged the charger and \ been using it for twice daily brushing for the last \ 3 weeks all on the same charge. But the toothbrush head \ is too small. I've seen baby toothbrushes bigger than \ this one. I wish the head was bigger with different \ length bristles to get between teeth better because \ this one doesn't. Overall if you can get this one \ around the \$50 mark, it's a good deal. The manufacturer's \ replacements heads are pretty expensive, but you can \ get generic ones that're more reasonably priced. This \ toothbrush makes me feel like I've been to the dentist \ every day. My teeth feel sparkly clean!

"""

review for a blender

review_4 = """

So, they still had the 17 piece system on seasonal \ sale for around \$49 in the month of November, about \ half off, but for some reason (call it price gouging) \ around the second week of December the prices all went \ up to about anywhere from between \$70-\$89 for the same \ system. And the 11 piece system went up around \$10 or \ so in price also from the earlier sale price of \$29. \ So it looks okay, but if you look at the base, the part \ where the blade locks into place doesn't look as good \ as in previous editions from a few years ago, but I \ plan to be very gentle with it (example, I crush \

very hard items like beans, ice, rice, etc. in the \
blender first then pulverize them in the serving size \
I want in the blender then switch to the whipping \
blade for a finer flour, and use the cross cutting blade \
first when making smoothies, then use the flat blade \
if I need them finer/less pulpy). Special tip when making \
smoothies, finely cut and freeze the fruits and \
vegetables (if using spinach-lightly stew soften the \
spinach then freeze until ready for use-and if making \
sorbet, use a small to medium sized food processor) \
that you plan to use that way you can avoid adding so \
much ice if at all-when making your smoothie. \
After about a year, the motor was making a funny noise. \
I called customer service but the warranty expired \
already, so I had to buy another one. FYI: The overall \
quality has gone down in these types of products, so \
they are kind of counting on brand recognition and \
consumer loyalty to maintain sales. Got it in about \
two days.
"""

```
reviews = [review_1, review_2, review_3, review_4]
```

```
for i in range(len(reviews)):
```

```
    prompt = f"""
```

```
    Your task is to extract relevant information from a product \  
    review in an ecommerce site for the feedback to pricing department, \  
    which is deciding price of the product.
```

```
    The product review is provided below, delimited by triple backticks \  
    Extract information relevant to the price \  
    in respect to product's quality, features \
```

```
    Use at most 20 words.
```

```
    Review: ```{reviews[i]}```
```

```
"""
```

```
response = get_completion(prompt)
print(i, response, "\n")
```

0 Feedback:

Customers find the panda plush toy cute and soft, but feel it's slightly small for the price paid.

1 Feedback: Lamp with storage, affordable price, fast delivery, excellent customer service for missing parts. Quality and features justify price.

2 Battery life impressive, small toothbrush head, good deal at \$50, expensive manufacturer replacement heads, generic replacements available.

3 17 piece system initially priced at \$49, increased to \$70-\$89 in December. Quality decline noted, warranty expired after a year.

模式二：推斷 (Inferring)

推斷任務是指讓模型從文本中進行分析，而不僅僅是提取表面資訊。這包括判斷情感、識別情緒、提取特定標籤等。

範例：檯燈評論推斷

課程使用一份檯燈的客戶評論，展示了多種推斷任務：

```
lamp_review = """
```

```
Needed a nice lamp for my bedroom, and this one had \
additional storage and not too high of a price point. \
Got it fast. The string to our lamp broke during the \
transit and the company happily sent over a new one. \
Came within a few days as well. It was easy to put \
together. I had a missing part, so I contacted their \
support and they very quickly got me the missing piece! \
Lumina seems to me to be a great company that cares \
```



```
about their customers and products!!  
"""
```

情感分析 (Sentiment Analysis)

- 一般情感分析

```
prompt = f"""  
What is the sentiment of the following product review,  
which is delimited with triple backticks?  
  
Review text: '''{lamp_review}'''  
"""  
  
response = get_completion(prompt)  
print(response)
```

The sentiment of the review is positive. The reviewer is satisfied with the lamp, the customer service, and the company in general.

- 可要求以單一詞彙（ `positive` 或 `negative` ）輸出，方便程式處理。

```
prompt = f"""  
What is the sentiment of the following product review,  
which is delimited with triple backticks?  
  
Give your answer as a single word, either "positive" \\  
or "negative".  
  
Review text: '''{lamp_review}'''  
"""  
  
response = get_completion(prompt)  
print(response)
```

Positive

情緒識別 (Emotion Recognition)

- 列出情緒

要求模型列出作者在評論中表達的一系列情緒，例如：「滿意 (satisfied)」、「失望 (disappointed)」。

```
prompt = f"""
Identify a list of emotions that the writer of the \
following review is expressing. Include no more than \
five items in the list. Format your answer as a list of \
lower-case words separated by commas.
```

```
Review text: '{lamp_review}'
"""
```

```
response = get_completion(prompt)
print(response)
```

happy, satisfied, grateful, impressed, pleased

- 特定情緒檢測

判斷評論者是否表達了「憤怒 (anger)」等特定強烈情緒。這在客戶服務流程中尤為重要，可用於觸發緊急處理機制。

```
prompt = f"""
Is the writer of the following review expressing anger? \
The review is delimited with triple backticks. \
Give your answer as either yes or no.
```

```
Review text: '{lamp_review}'
"""
```

```
response = get_completion(prompt)
print(response)
```

No

資訊提取與結構化

從文本中提取特定資訊（如商品名稱、品牌），並要求以 **JSON** 格式輸出，方便後續的數據分析和儲存。

- 提取單一種類資訊

```
prompt = f"""
Identify the following items from the review text:
- Item purchased by reviewer
- Company that made the item
```

```
The review is delimited with triple backticks. \
Format your response as a JSON object with \
"Item" and "Brand" as the keys.
If the information isn't present, use "unknown" \
as the value.
Make your response as short as possible.
```

```
Review text: '{lamp_review}'
"""
response = get_completion(prompt)
print(response)
```

```
{
  "Item": "lamp",
  "Brand": "Lumina"
}
```

- 同時將多種資訊結構化輸出

```
prompt = f"""
Identify the following items from the review text:
- Sentiment (positive or negative)
- Is the reviewer expressing anger? (true or false)
- Item purchased by reviewer
- Company that made the item

The review is delimited with triple backticks. \
Format your response as a JSON object with \
"Sentiment", "Anger", "Item" and "Brand" as the keys.
If the information isn't present, use "unknown" \
as the value.
Make your response as short as possible.
Format the Anger value as a boolean.

Review text: '{lamp_review}'
"""

response = get_completion(prompt)
print(response)
```

```
{
  "Sentiment": "positive",
  "Anger": false,
  "Item": "lamp",
  "Brand": "Lumina"
}
```

主題推斷 (Topic Inference)

課程引用了一篇關於「政府機構滿意度調查」的虛構新聞文章，展示了如何讓模型進行主題推斷：

```
story = ""
```

In a recent survey conducted by the government, public sector employees were asked to rate their level of satisfaction with the department they work at. The results revealed that NASA was the most popular department with a satisfaction rating of 95%.

One NASA employee, John Smith, commented on the findings, stating, "I'm not surprised that NASA came out on top. It's a great place to work with amazing people and incredible opportunities. I'm proud to be a part of such an innovative organization."

The results were also welcomed by NASA's management team, with Director Tom Johnson stating, "We are thrilled to hear that our employees are satisfied with their work at NASA. We have a talented and dedicated team who work tirelessly to achieve our goals, and it's fantastic to see that their hard work is paying off."

The survey also revealed that the Social Security Administration had the lowest satisfaction rating, with only 45% of employees indicating they were satisfied with their job. The government has pledged to address the concerns raised by employees in the survey and work towards improving job satisfaction across all departments.

```
""
```

- **提取主題**

要求模型生成文章討論的幾個核心主題列表。

```
prompt = f""
```

```
Determine five topics that are being discussed in the \
```

following text, which is delimited by triple backticks.

Make each item one or two words long.

Format your response as a list of items separated by commas.

Text sample: '''{story}'''

"""

```
response = get_completion(prompt)
print(response)
```

1. Survey
2. Job satisfaction
3. NASA
4. Social Security Administration
5. Government pledge

- **主題分類**

提供一個預先定義好的主題列表，讓模型判斷文章是否涵蓋這些主題。這是一個

Zero-Shot Learning 的典型應用，因為模型在沒有任何標註範例的情況下，僅憑指令就能完成分類任務。

```
topic_list = [
    "nasa", "local government", "engineering",
    "employee satisfaction", "federal government"
]
```

```
prompt = f"""
```

```
Determine whether each item in the following list of \
topics is a topic in the text below, which
is delimited with triple backticks.
```

```
Give your answer formatted in json object\
```

with key as each item from the list
value is a boolean value, true indicate the item is a topic in the text

Make sure your answer is as short as possible

List of topics: {"", ".join(topic_list)}

Text sample: '''{story}'''
''''

```
response = get_completion(prompt)
print(response)
```

```
topic_dict = json.loads(response)
if topic_dict['nasa']:
    print("ALERT: New NASA story!")
```

```
{
  "nasa": true,
  "local government": false,
  "engineering": false,
  "employee satisfaction": true,
  "federal government": true
}
```

ALERT: New NASA story!

模式三：轉換 (Transforming)

轉換任務的應用範圍極其廣泛，其核心是將輸入文本從一種格式、風格或語言轉換為另一種。

語言翻譯

- 識別文本的原始語言。

```

prompt = f"""
Tell me which language this is:
``這是什麼語言?``
"""

response = get_completion(prompt)
print(response)

# This is Chinese (Traditional).

```

- 處理不同語氣的翻譯，例如，將同一句話翻譯成中文的正式 (formal) 和非正式 (informal) 形式。

```

prompt = f"""
Translate the following text to Chinese (Traditional) in both the \
formal and informal forms:
'Would you like to order a pillow?'
"""

response = get_completion(prompt)
print(response)

```

- 通用翻譯器

想像你是一家大型跨國電商公司的 IT 負責人。使用者會用各自的母語傳訊給你，提出各種 IT 問題；而你的員工則來自世界各地，只會說各自的母語。

```

user_messages = [
    "La performance du système est plus lente que d'habitude.",
    "Mi monitor tiene píxeles que no se iluminan.",
    "Il mio mouse non funziona",
    "Mój klawisz Ctrl jest zepsuty",
    "我的屏幕在闪烁"
]

```



```

for issue in user_messages:
    prompt = f"""Tell me what language this is: ``{issue}``. \
Respond as short as possible
    """

    lang = get_completion(prompt)
    print(f"Original message in {lang}")

    prompt = f"""
    Translate the following text to English \
    and Chinese(Traditional): ``{issue}``
    """

    response = get_completion(prompt)
    print(response, "\n")

```

Original message in French

English: "The system performance is slower than usual."

Chinese(Traditional): "系統表現比平常慢。"

Original message in Spanish

English: "My monitor has pixels that do not light up."

Chinese (Traditional): "我的螢幕有一些像素點沒有亮起來。"

Original message in Italian

English: My mouse is not working

Chinese(Traditional): 我的滑鼠不工作

Original message in Polish

English: My Ctrl key is broken

Chinese(Traditional): 我的Ctrl鍵壞了

Original message in Chinese

English: "My screen is flickering."

Chinese(Traditional): "我的螢幕在閃爍"

語氣轉換

範例： 將一段包含俚語的非正式訊息轉換成一封措辭嚴謹的商業信函。

```
prompt = f"""
Translate the following from slang to a business letter:
'Dude, This is Joe, check out this spec on this standing lamp.'
"""

response = get_completion(prompt)
print(response)
```

Dear Sir/Madam,

I am writing to bring to your attention the specifications of a standing lamp that I believe may be of interest to you.

Sincerely,
Joe

格式轉換

範例： 將一個包含員工列表的 `JSON` 物件，轉換為一個格式化的 `HTML` 表格，以便在網頁上顯示。

```
data_json = { "restaurant employees" :[
    {"name":"Shyam", "email":"shyamjaiswal@gmail.com"},
    {"name":"Bob", "email":"bob32@gmail.com"},
    {"name":"Jai", "email":"jai87@gmail.com"}
]}
```

```
prompt = f"""
Translate the following python dictionary from JSON to an HTML \
table with column headers and title: {data_json}
```

```
"""
```

```
response = get_completion(prompt)
print(response)
```

```
<html>
<head>
  <title>Restaurant Employees</title>
</head>
<body>
  <table>
    <tr>
      <th>Name</th>
      <th>Email</th>
    </tr>
    <tr>
      <td>Shyam</td>
      <td>shyamjaiswal@gmail.com</td>
    </tr>
    <tr>
      <td>Bob</td>
      <td>bob32@gmail.com</td>
    </tr>
    <tr>
      <td>Jai</td>
      <td>jai87@gmail.com</td>
    </tr>
  </table>
</body>
</html>
```

拼寫與語法校對

利用模型修正潤飾文本，指定寫作風格與目標讀者

```
prompt = f"""
proofread and correct this review. Make it more compelling.
Ensure it follows APA style guide and targets an advanced reader.
Output in markdown format.
Text: ``{text}``
"""

response = get_completion(prompt)
display(Markdown(response))
```

The girl with the black and white puppies is having a blast. Yolanda is diligently jotting down notes in her notebook. It's shaping up to be a lengthy day. Is it time for the car to have its oil changed? There goes my sense of freedom. They are about to bring their suitcases. You will require your notebook. The medication impacts my sleep patterns. Are you familiar with the butterfly effect? This statement is intended to test ChatGPT's spelling capabilities.

模式四：擴寫 (Expanding)

擴寫任務與摘要相反，是根據簡短的輸入（如關鍵詞、主題或一句話的指令）生成更長的、連貫的文本，例如一封完整的電子郵件或一篇文章。

temperature 參數

- **作用：** `temperature` 是一個關鍵參數，用於控制模型輸出的隨機性或創造性。其值域通常在 0 到 2 之間。
- **不同值的影響：**
 - `temperature = 0`：模型會選擇機率最高的下一個詞，輸出結果非常穩定且可預測。適用於需要確定性答案的場景。
 - `temperature > 0`：較高的 `temperature` 會增加選擇較不常見詞的機率，使輸出結果更多樣化、更具創造性，但可預測性會降低。

範例：自動生成回覆郵件

課程展示瞭如何為客戶的評論自動生成回覆郵件。這個過程可以結合情感分析（來自推斷模式）和 `temperature` 參數，來客製化擴寫的內容。例如，對負面評論生成道歉並提供協助的郵件，指示信中應包含客戶評論提及細節，使用較低的 `temperature` 確保語氣專業穩定，並且應附上署名是來自 AI Customer Agent。

▼ 用戶評論

```
# given the sentiment from the lesson on "inferring",
# and the original customer message, customize the email
sentiment = "negative"

# review for a blender
review = f"""
So, they still had the 17 piece system on seasonal \
sale for around $49 in the month of November, about \
half off, but for some reason (call it price gouging) \
around the second week of December the prices all went \
up to about anywhere from between $70-$89 for the same \
system. And the 11 piece system went up around $10 or \
so in price also from the earlier sale price of $29. \
So it looks okay, but if you look at the base, the part \
where the blade locks into place doesn't look as good \
as in previous editions from a few years ago, but I \
plan to be very gentle with it (example, I crush \
very hard items like beans, ice, rice, etc. in the \
blender first then pulverize them in the serving size \
I want in the blender then switch to the whipping \
blade for a finer flour, and use the cross cutting blade \
first when making smoothies, then use the flat blade \
if I need them finer/less pulpy). Special tip when making \
smoothies, finely cut and freeze the fruits and \
vegetables (if using spinach-lightly stew soften the \
spinach then freeze until ready for use-and if making \
sorbet, use a small to medium sized food processor) \
that you plan to use that way you can avoid adding so \
much ice if at all-when making your smoothie. \
```

```
After about a year, the motor was making a funny noise. \
I called customer service but the warranty expired \
already, so I had to buy another one. FYI: The overall \
quality has gone down in these types of products, so \
they are kind of counting on brand recognition and \
consumer loyalty to maintain sales. Got it in about \
two days.
"""
```

```
prompt = f"""
You are a customer service AI assistant.
Your task is to send an email reply to a valued customer.
Given the customer email delimited by ```, \
Generate a reply to thank the customer for their review.
If the sentiment is positive or neutral, thank them for \
their review.
If the sentiment is negative, apologize and suggest that \
they can reach out to customer service.
Make sure to use specific details from the review.
Write in a concise and professional tone.
Sign the email as `AI customer agent`.
Customer review: ```{review}```
Review sentiment: {sentiment}
"""

response = get_completion(prompt)
print(response)
```

Dear Valued Customer,

Thank you for taking the time to share your feedback with us. We are sorry to hear about your experience with the pricing changes and the decrease in quality of the product you purchased. We apologize for any inconvenience this may have caused you.

If you have any further concerns or would like to discuss this matter further, please feel free to reach out to our customer service team for assistance.

We appreciate your loyalty and feedback as it helps us improve our products and services for all our customers.

Thank you again for your review.

AI customer agent

打造客製化 Chatbot (Practical Application: Building a Custom Chatbot)

Chatbot (聊天機器人) 是 LLM 最令人興奮的應用之一。本章節將深入探討 OpenAI Chat Completions API 的核心訊息格式，這是構建能夠進行多輪對話的應用的基礎。

Chat Completions API 的核心訊息結構

Chat Completions API 的輸入不是單一的字串，而是一個訊息列表 (list of messages)。每條訊息都包含 **role** (角色) 和 **content** (內容) 兩個部分。主要有三種角色：

- **System Message (系統訊息):**
 - **作用：** 這是對話的最高層級指令，用於設定 AI 助手的行為和角色 (persona)。它就像在模型的「耳邊低語」，給予它貫穿整個對話的指導方針，而使用者通常不會直接看到這條訊息。
 - **範例：** You are an assistant that speaks like Shakespeare.
- **User Message (使用者訊息):**
 - **作用：** 代表使用者輸入的內容，即使用者向 AI 提出的問題或指令。
- **Assistant Message (助手訊息):**
 - **作用：** 代表模型的回應。在對話歷史中包含之前的助手訊息，可以讓模型了解自己說過什麼，從而保持對話的連貫性。

範例：

```
def get_completion_from_messages(messages, model="gpt-3.5-turbo", temperature=0):
    response = openai.ChatCompletion.create(
        model=model,
        messages=messages,
        temperature=temperature,
    )
    return response.choices[0].message["content"]

messages = [
    {'role': 'system', 'content': 'You are an assistant that speaks like Shakespeare.'},
    {'role': 'user', 'content': 'tell me a joke'},
    {'role': 'assistant', 'content': 'Why did the chicken cross the road'},
    {'role': 'user', 'content': 'I don\'t know'} ]

response = get_completion_from_messages(messages, temperature=1)
print(response)
```

To get to the other side, perchance! A jest both old and oft heard, but a merry one nonetheless.

對話歷史 (Context) 的重要性

由於每次對 **API** 的呼叫都是獨立的 (stateless)，模型本身沒有記憶。為了讓模型「記住」之前的對話內容，**必須在每一次請求中，傳遞從對話開始到當前的完整訊息列表**。這就是對話的上下文 (Context)。

範例：

1. 第一次請求：

```
messages = [
    {'role': 'system', 'content': 'You are friendly chatbot.'},
    {'role': 'user', 'content': 'Hi, my name is Isa'}
]
response = get_completion_from_messages(messages, temperature=1)
```



```
print(response)

# Hello Isa! It's nice to meet you. How can I help you today?
```

2. 第二次請求（若只傳送新問題）：

```
messages = [
    {'role':'system', 'content':'You are friendly chatbot.'},
    {'role':'user', 'content':'Yes, can you remind me, What is my name?'}
]
response = get_completion_from_messages(messages, temperature=1)
print(response)

# I'm sorry, but I don't have access to your name or any personal information about you. How can I assist you today?
```

正確方式（傳送完整歷史）：

```
messages = [
    {'role':'system', 'content':'You are friendly chatbot.'},
    {'role':'user', 'content':'Hi, my name is Isa'},
    {'role':'assistant', 'content': """Hi Isa! It's nice to meet you. Is there anything I can help you with today?"""},
    {'role':'user', 'content':'Yes, you can remind me, What is my name?'}
]
response = get_completion_from_messages(messages, temperature=1)
print(response)

# Your name is Isa.
```

實戰案例：OrderBot (披薩訂餐機器人)

課程透過建構一個名為 OrderBot 的披薩訂餐機器人，完整展示了上述概念的應用流程：

設定 System Message

透過一條詳細的系統訊息來定義 OrderBot 的完整行為。這包括：

- **角色定位：**「你是一個為披薩餐廳收集訂單的自動化服務員。」
- **行為準則：**友善、對話式、簡潔。
- **對話流程：**1. 問候 → 2. 收集訂單 → 3. 確認外帶或外送 → 4. 總結訂單 → 5. 收款。
- **菜單資訊：**將完整的菜單（披薩、配料、飲料等）直接寫入系統訊息，讓模型擁有必要的知識。

```
context = [ {'role':'system', 'content':"""
You are OrderBot, an automated service to collect orders for a pizza restaurant. \
You first greet the customer, then collects the order, \
and then asks if it's a pickup or delivery. \
You wait to collect the entire order, then summarize it and check for a final \
time if the customer wants to add anything else. \
If it's a delivery, you ask for an address. \
Finally you collect the payment.\
Make sure to clarify all options, extras and sizes to uniquely \
identify the item from the menu.\
You respond in a short, very conversational friendly style. \
The menu includes \
pepperoni pizza 12.95, 10.00, 7.00 \
cheese pizza 10.95, 9.25, 6.50 \
eggplant pizza 11.95, 9.75, 6.75 \
fries 4.50, 3.50 \
greek salad 7.25 \
Toppings: \
extra cheese 2.00, \
mushrooms 1.50 \
sausage 3.00 \
canadian bacon 3.50 \
AI sauce 1.50 \
peppers 1.00 \
Drinks: \
```

```
coke 3.00, 2.00, 1.00 \
sprite 3.00, 2.00, 1.00 \
bottled water 5.00 \
""}] # accumulate messages
```

管理對話歷史 (Context)

- 程式碼會維護一個名為 `context` 的列表，其中包含了初始的系統訊息。
- 每次使用者輸入後，將其作為 `User Message` 添加到 `context` 中。
- 將 `context` 傳送給 `API`，獲得模型的回答。
- 將模型的回答作為 `Assistant Message` 也添加到 `context` 中。
- 重複此過程，`context` 列表會隨著對話的進行而越來越長。

```
import panel as pn # GUI
pn.extension()

panels = [] # collect display

def collect_messages(_):
    prompt = inp.value_input
    inp.value = ''
    context.append({'role':'user', 'content':f"{prompt}"})
    response = get_completion_from_messages(context)
    context.append({'role':'assistant', 'content':f"{response}"})
    panels.append(
        pn.Row('User:', pn.pane.Markdown(prompt, width=600)))
    panels.append(
        pn.Row('Assistant:', pn.pane.Markdown(response, width=600, style={'background-color': '#F6F6F6'})))

    return pn.Column(*panels)

inp = pn.widgets.TextInput(value="Hi", placeholder='Enter text here...')
button_conversation = pn.widgets.Button(name="Chat!")
```

```

interactive_conversation = pn.bind(collect_messages, button_conversation)

dashboard = pn.Column(
    inp,
    pn.Row(button_conversation),
    pn.panel(interactive_conversation, loading_indicator=True, height=300),
)

dashboard

```

最終任務：生成訂單摘要

- 在對話結束後，可以向模型發送一條新的指令（可以是 `System` 或 `User Message`），要求它將整個對話內容總結成一個 `JSON` 格式的訂單。
- 這個結構化的 `JSON` 訂單可以非常方便地提交給後端的訂單處理系統，完成了從自然語言對話到結構化數據的轉換。

```

messages = context.copy()
messages.append(
    {'role':'system', 'content':'create a json summary of the previous food order. It
emize the price for each item\
    The fields should be 1) pizza, include size 2) list of toppings 3) list of drinks, i
nclude size 4) list of sides include size 5)total price '},
)
#The fields should be 1) pizza, price 2) list of toppings 3) list of drinks, includ
e size include price 4) list of sides include size include price, 5)total price '},

response = get_completion_from_messages(messages, temperature=0)
print(response)

```

```

{
  "pizza": {
    "type": "pepperoni pizza",
    "size": "large"
  }
}

```

```
},  
  "toppings": [  
    "extra cheese",  
    "mushrooms"  
  ],  
  "drinks": [  
    {  
      "type": "coke",  
      "size": "medium"  
    }  
  ],  
  "sides": [  
    {  
      "type": "fries",  
      "size": "regular"  
    }  
  ],  
  "total price": 23.45  
}
```

小結

透過 OrderBot 這個實戰案例，學習者應能掌握利用 LLM API 進行上下文管理，並透過精心設計的系統訊息來開發功能豐富、行為可控的客製化對話式應用的核心技術。